

SpringBoot

How to Create Custom Methods in Spring Data JPA :-

Custom methods in Spring Data JPA can be created in two main ways:

1. Method Naming Convention (Query Derivation)
2. Using @Query Annotation

1. Custom Method Using Method Naming Convention :-

Steps:-

1. Create an Entity class that represents a database table.
2. Add fields in the entity that map to table columns.
3. Create a Repository interface for the entity.
4. Extend the Repository interface with JpaRepository.
5. Inside the repository, write a custom method following naming rules.
6. Start the method name with findBy, readBy, or getBy.
7. Add the entity field name after the keyword.
8. Add conditions like And, Or, GreaterThan, Like, etc., if needed.
9. Spring automatically creates the SQL query based on the method name.
10. Use the custom method in the Service or Controller to fetch filtered data.

Example:

```
List<Student> findByEmail(String city);
```

Spring generates query internally:

```
SELECT * FROM student WHERE email = ?;
```

Basic Structure :

Action + By + Field + Condition

Examples:

1. findByName
2. findByEmail
3. findByPassword
4. findByAddress

2. Custom Method Using @Query Annotation steps :-

Steps:-

1. Create an Entity class to map the database table.
2. Create a Repository interface for the entity.
3. Extend the Repository with JpaRepository.
4. Define a method inside the repository for the custom query.
5. Above the method, use the @Query annotation.
6. Write JPQL or SQL inside the @Query annotation.
7. If using SQL, set nativeQuery = true.
8. Use @Param to bind method parameters to query parameters.
9. Spring executes the given JPQL/SQL instead of generating one automatically.
10. Call this custom method from the Service or Controller to get data.

JPQL Example

```
@Query("SELECT s FROM Student s WHERE s.city = :city")  
List<Student> getStudentsByCity(String city);
```

Native SQL Example

```
@Query(value = "SELECT * FROM student WHERE city = :city", nativeQuery = true)  
List<Student> getByCityNative(String city);
```

Advantages of Custom Method :-

1. Custom methods allow writing queries without SQL or JPQL.
2. Method names make the code simple, clean, and readable.
3. Faster development because Spring generates queries automatically.
4. Easy to maintain as logic is clearly expressed through method names.

Disadvantages of Custom Method :-

1. Method names become too long and hard to read for complex queries.
2. Not suitable for advanced queries like joins or subqueries.
3. Limited flexibility because everything depends on naming rules.
4. Complex logic cannot be expressed effectively using naming conventions.

Login Form Task :-

login.jsp -

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css" rel="stylesheet"
    integrity="sha384-sRIL4kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqkuIAvNwLzeN8tE5YBuJZqJLB" crossorigin="anonymous">
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.min.js"
    integrity="sha384-FKyoEForCGLyvwx9Hj09JcYn3nv7wiPVLz7YYwJrWVcXK/BmnVDxM+D2scQbITxI" crossorigin="anonymous"></script>
</head>
<body>

    <%@ include file="header.jsp" %>

    <div class="row">
        <div class="col-sm-4"></div>
        <div class="col-sm-4">
            <h1>Login Form</h1><br>

            <form action="logincheck" method="post">
                <input type="email" name="email" placeholder="Enter your email" class="form-control mb-2">
                <input type="password" name="password" placeholder="Enter your password" class="form-control mb-2">
                <button type="submit" class="btn btn-success">Submit</button>
            </form>
        </div>
        <div class="col-sm-4"></div>
    </div>

</body>
</html>
```

dashboard.jsp –

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<meta charset="UTF-8">
<title>Insert title here</title>
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css" rel="stylesheet"
    integrity="sha384-sRIL4kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqkuIAvNwLzeN8tE5YBuJZqJLB" crossorigin="anonymous">
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.min.js"
    integrity="sha384-FKyoEForCGLyvwx9Hj09JcYn3nv7wiPVLz7YYwJrWVcXK/BmnVDxM+D2scQbITxI" crossorigin="anonymous"></script>
</head>
<body>

    <%@ include file="header.jsp" %>

    <h2>Dashboard Page</h2>

    <p>${user.fullname}</p>
    <p>${user.email}</p>
    <p>${user.address}</p>

    <br><br>

    <a href="/Logout">Logout</a>

</body>
</html>
```

MyController.java –

```
package com.example.demo;

import java.util.List;
import java.util.Optional;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestParam;

import jakarta.servlet.http.HttpSession;

@Controller
public class MyController {

    @Autowired
    StudentService ss;

    @GetMapping("/")
    public String home()
    {
        return "home"; //home.jsp
    }

    @GetMapping("/register")
    public String register()
    {
        return "register"; //register.jsp
    }

    @GetMapping("/viewdata")
    public String viewdata(Model m) {
        List<Student> list = ss.getAllStudents();
        m.addAttribute("list", list);
        return "viewdata"; //viewdata.jsp
    }

    @GetMapping("/login")
    public String login()
    {
        return "login"; //login.jsp
    }

    @PostMapping("/saveform")
    public String saveform(@ModelAttribute Student s) {
        ss.registration(s);
        return "redirect:viewdata";
    }

    @GetMapping("/delete/{id}")
    public String delete(@PathVariable Long id) {
        try {
            ss.delete(id);
        } catch (Exception e) {
            System.out.println(e);
        }
        return "redirect:/viewdata";
    }

    @GetMapping("/profileedit/{id}")
    public String profileedit(@PathVariable Long id, Model m) {
        Optional<Student> s = ss.getStudentById(id);
        m.addAttribute("student", s.get());
        return "edit"; //edit.jsp
    }
}
```

```

@PostMapping("/saveprofile/{id}")
public String saveprofile(@PathVariable Long id, @ModelAttribute Student s) {
    ss.profileupdate(id,s);
    return "redirect:/viewdata";
}

@PostMapping("/logincheck")
public String logincheck(@RequestParam String email,@RequestParam String password, HttpSession session,Model m) {
    Student s = ss.logincheck(email, password);
    try {
        session.setAttribute("user", s);    //session start
        m.addAttribute("user", s);
        System.out.println("Login Successfully");
        return "redirect:/dashboard";
    } catch(Exception e) {
        System.out.println(e);
        return "redirect:/login";
    }
}

@GetMapping("/dashboard")
public String dashboard(HttpSession session,Model m) {
    Student s = (Student)session.getAttribute("user");    //session get
    m.addAttribute("user", s);
    if (s == null) {
        return "redirect:/login";
    }
    return "dashboard";    //dashboard.jsp
}

@GetMapping("/logout")
public String logout(HttpSession session) {
    session.invalidate();    //session end
    return "redirect:/login";
}
}

```

StudentRepo.java –

```
package com.example.demo;

import java.util.Optional;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface StudentRepo extends JpaRepository<Student, Long> {

    Optional<Student>findByEmail(String email);

}
```

StudentService.java –

```
package com.example.demo;

import java.util.List;
import java.util.Optional;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

@Service
public class StudentService {

    @Autowired
    StudentRepo sr;

    public void registration(Student s)
    {
        sr.save(s);        //insert
    }

    public List<Student> getallstudents(){
        return sr.findAll();    //select * from student
    }

    public void delete(Long id) {
        //sr.deleteById(id);

        if(sr.existsById(id)) {
            sr.deleteById(id);
        }else {
            throw new RuntimeException("Id not found");
        }
    }

    public Optional<Student> getStudentById(Long id) {
        return sr.findById(id);
    }

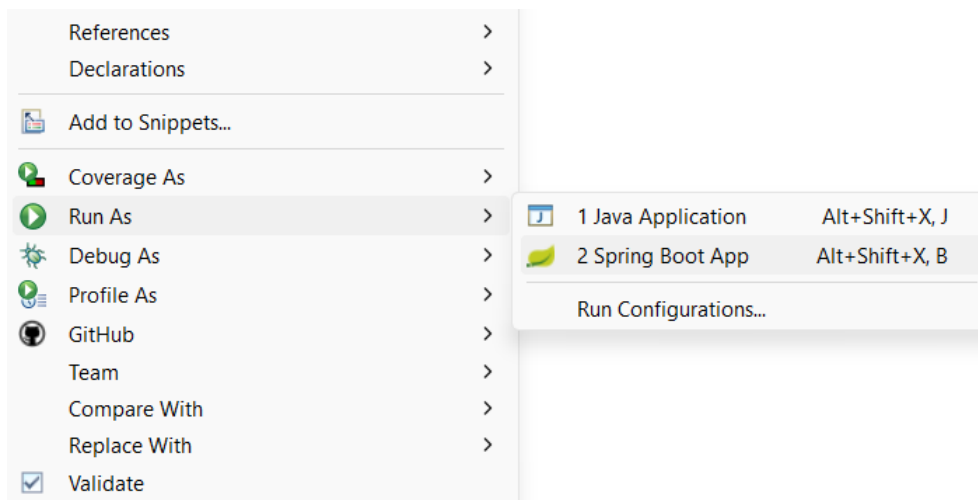
    public void profileupdate(Long id, Student s) {
        Student stud = sr.findById(id).orElseThrow(()->new RuntimeException("Id not found"));
        stud.setFullname(s.getFullname());
        stud.setEmail(s.getEmail());
        stud.setPassword(s.getPassword());
        stud.setAddress(s.getAddress());
        sr.save(stud);
    }

    public Student logincheck(String email,String password) {
        Student s = sr.findByEmail(email).orElseThrow(()->new RuntimeException("Id not found"));

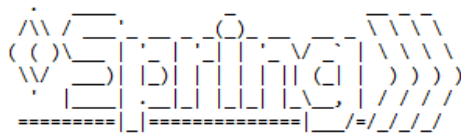
        if(!s.getPassword().equals(password)) {
            throw new RuntimeException("Password not found");
        }

        return s;
    }
}
```

Run the Application –



Console Based -



:: Spring Boot :: (v4.0.0)

```
2025-12-03T20:24:41.755+05:30 INFO 20928 --- [Spring_Boot_Sample_Project2] [ restartedMain]
\sagar\workspace\Spring_Boot_Sample_Project2\target\classes started by sagar in C:\Us
2025-12-03T20:24:41.763+05:30 INFO 20928 --- [Spring_Boot_Sample_Project2] [ restartedMain]
2025-12-03T20:24:41.897+05:30 INFO 20928 --- [Spring_Boot_Sample_Project2] [ restartedMain]
disable
2025-12-03T20:24:41.897+05:30 INFO 20928 --- [Spring_Boot_Sample_Project2] [ restartedMain]
'DEBUG'
2025-12-03T20:24:43.487+05:30 INFO 20928 --- [Spring_Boot_Sample_Project2] [ restartedMain]
2025-12-03T20:24:43.613+05:30 INFO 20928 --- [Spring_Boot_Sample_Project2] [ restartedMain]
2025-12-03T20:24:44.804+05:30 INFO 20928 --- [Spring_Boot_Sample_Project2] [ restartedMain]
2025-12-03T20:24:44.841+05:30 INFO 20928 --- [Spring_Boot_Sample_Project2] [ restartedMain]
2025-12-03T20:24:44.842+05:30 INFO 20928 --- [Spring_Boot_Sample_Project2] [ restartedMain]
2025-12-03T20:24:45.448+05:30 INFO 20928 --- [Spring_Boot_Sample_Project2] [ restartedMain]
logger for a complete list of JARs that were scanned but no TLDs were found in them. Skipping
2025-12-03T20:24:45.470+05:30 INFO 20928 --- [Spring_Boot_Sample_Project2] [ restartedMain]
2025-12-03T20:24:45.883+05:30 INFO 20928 --- [Spring_Boot_Sample_Project2] [ restartedMain]
2025-12-03T20:24:46.091+05:30 INFO 20928 --- [Spring_Boot_Sample_Project2] [ restartedMain]
2025-12-03T20:24:47.532+05:30 INFO 20928 --- [Spring_Boot_Sample_Project2] [ restartedMain]
2025-12-03T20:24:47.635+05:30 INFO 20928 --- [Spring_Boot_Sample_Project2] [ restartedMain]
2025-12-03T20:24:48.301+05:30 INFO 20928 --- [Spring_Boot_Sample_Project2] [ restartedMain]
2025-12-03T20:24:48.306+05:30 INFO 20928 --- [Spring_Boot_Sample_Project2] [ restartedMain]
2025-12-03T20:24:48.535+05:30 INFO 20928 --- [Spring_Boot_Sample_Project2] [ restartedMain]
Database JDBC URL [jdbc:postgresql://localhost:5433/springbootdb]
Database driver: PostgreSQL JDBC Driver
Database dialect: PostgreSQLDialect
Database version: 18.0
Default catalog/schema: springbootdb/public
Autocommit mode: undefined/unknown
Isolation level: READ_COMMITTED [default READ_COMMITTED]
JDBC fetch size: none
Pool: DatasourceConnectionProviderImpl
Minimum pool size: undefined/unknown
Maximum pool size: undefined/unknown
2025-12-03T20:24:50.744+05:30 INFO 20928 --- [Spring_Boot_Sample_Project2] [ restartedMain]
JTA platform integration)
2025-12-03T20:24:50.922+05:30 INFO 20928 --- [Spring_Boot_Sample_Project2] [ restartedMain]
2025-12-03T20:24:51.150+05:30 INFO 20928 --- [Spring_Boot_Sample_Project2] [ restartedMain]
2025-12-03T20:24:51.612+05:30 WARN 20928 --- [Spring_Boot_Sample_Project2] [ restartedMain]
performed during view rendering. Explicitly configure spring.jpa.open-in-view to disable this
2025-12-03T20:24:52.445+05:30 INFO 20928 --- [Spring_Boot_Sample_Project2] [ restartedMain]
2025-12-03T20:24:52.456+05:30 INFO 20928 --- [Spring_Boot_Sample_Project2] [ restartedMain]
```

2025-12-03T20:35:54.118+05:30 WARN 20928 --- [Spring_Boot_Sample_Project2]
[[org.springframework.web.HttpRequestMethodNotSupportedException](#): Request method not supported] Hibernate:
select

```
    s1_0.id,  
    s1_0.address,  
    s1_0.email,  
    s1_0.fullname,  
    s1_0.password  
from  
    student s1_0  
where  
    s1_0.email=?
```

Web Based –

<http://localhost:8080/login>

A screenshot of a web browser showing the login page. The address bar displays 'localhost:8080/login'. The page has a navigation bar with links: 'Navbar', 'Home', 'Register', 'Students View Data', and 'Login'. The main heading is 'Login Form'. Below it, there are two input fields: the first contains the email 'sagarkate374@gmail.com' and the second contains a masked password '...'. A green 'Submit' button is located below the password field.

1. If the Email Id & Password are Correct then –

A screenshot of the dashboard page after a successful login. The address bar shows 'localhost:8080/dashboard'. The navigation bar remains the same. The main heading is 'Dashboard Page'. Below the heading, the user's name 'Sagar Kate' is displayed, followed by their email 'sagarkate374@gmail.com' and their role 'Satara'. At the bottom, there is a blue link labeled 'Logout'.

After logout then –

A screenshot of the login page after logging out. The address bar shows 'localhost:8080/login'. The navigation bar is the same. The main heading is 'Login Form'. There are two empty input fields: 'Enter your email' and 'Enter your password'. A green 'Submit' button is at the bottom.

2. If the Email Id & Password are Incorrect then –

A screenshot of the login page showing an incorrect login attempt. The address bar shows 'localhost:8080/login'. The navigation bar is the same. The main heading is 'Login Form'. The input fields are empty, with placeholder text 'Enter your email' and 'Enter your password'. A green 'Submit' button is at the bottom.